



LeetCode 326: Power of Three

1. Problem Title & Link

Title: LeetCode 326: Power of Three

Link: <https://leetcode.com/problems/power-of-three/>

2. Problem Statement (Short Summary)

Given an integer n , return:

true

if n is a power of 3, otherwise return:

false

A number is a power of 3 if:

$n = 3^x$

for some integer:

$x \geq 0$

Example

Input:

$n = 27$

Output:

true

Because:

$27 = 3^3$

3. Examples (Input → Output)

Example 1

Input:

$n = 27$

Output:

true

Example 2

Input:

$n = 0$

Output:

false

Example 3



Input:

$n = 9$

Output:

true

Example 4

Input:

$n = 45$

Output:

false

4. Constraints

$-2^{31} \leq n \leq 2^{31} - 1$

Important:

Negative numbers cannot be powers of 3.

5. Core Concept (Pattern / Topic)

Math

Secondary Topics:

- Repeated Division
- Number Theory

6. Thought Process (Step-by-Step Explanation)

Understanding Power of Three

Sequence:

$$3^0 = 1$$

$$3^1 = 3$$

$$3^2 = 9$$

$$3^3 = 27$$

$$3^4 = 81$$

$$3^5 = 243$$

...

Valid numbers:

1

3

9

27



81
243
...

Key Observation

If a number is truly a power of 3:
Keep dividing by 3
Eventually:
It becomes 1

Example

Input:
27
Divide:
 $27 / 3 = 9$

 $9 / 3 = 3$

 $3 / 3 = 1$
Reached:
1
Therefore:
true

Example

Input:
45
Divide:
 $45 / 3 = 15$

 $15 / 3 = 5$
Now:
 $5 \% 3 \neq 0$
Cannot continue.
Return:
false



Key Insight

A power of 3 contains only factors of 3.

If any other factor appears:

Not a power of 3

7. Visual / Intuition Diagram (ASCII Diagram)

Valid Case

27

↓

9

↓

3

↓

1

Reached:

1

Return:

true

Invalid Case

45

↓

15

↓

5

Cannot divide further.

Return:

false

8. Pseudocode (Language Independent)

```
if n <= 0
    return false

while n % 3 == 0
    n = n / 3

return n == 1
```

9. Code Implementation

Python

```
class Solution:
    def isPowerOfThree(self, n: int) -> bool:
```



```
if n <= 0:
    return False

while n % 3 == 0:
    n //= 3

return n == 1
```

Java

```
class Solution {

    public boolean isPowerOfThree(int n) {

        if(n <= 0)
            return false;

        while(n % 3 == 0) {

            n /= 3;
        }

        return n == 1;
    }
}
```

10. Time & Space Complexity

Time Complexity

$O(\log_3 n)$

Each division reduces the number by a factor of 3.

Space Complexity

$O(1)$

Only a few variables are used.

11. Common Mistakes / Edge Cases

Mistake 1

Ignoring:

$n \leq 0$

Example:

$n = 0$

Output:

false

**Mistake 2**

Checking only once.

Wrong:

```
return n % 3 == 0
```

Example:

```
12 % 3 == 0
```

But:

12 is not a power of 3

Mistake 3

Using floating-point logarithms.

Can cause precision issues.

Edge Cases

n = 1

Input:

1

Output:

true

Because:

$1 = 3^0$

n = 3

Output:

true

n = -3

Output:

false

12. Detailed Dry Run

Input:

n = 81

Current n	Divisible by 3?	Next n
81	Yes	27



27	Yes	9
9	Yes	3
3	Yes	1

Reached:

1

Return:

true

Another Example

Input:

n = 45

Current n	Next n
45	15
15	5

Now:

$5 \% 3 \neq 0$

Return:

false

13. Common Use Cases (Real-Life / Interview)

Exponential Growth Models

Checking whether a value belongs to a geometric sequence.

Computer Science

Tree branching factors.

Mathematics

Factorization and divisibility.

Interview Purpose

Tests:

Mathematical Reasoning

Repeated Division

Number Theory



14. Common Traps (Important!)

Trap 1

Confusing:

Multiple of 3

with:

Power of 3

Example:

12

is multiple of 3.

But not:

3^x

Trap 2

Using logarithms.

Example:

`math.log(n,3)`

May fail due to floating-point precision.

Trap 3

Forgetting:

$1 = 3^0$

which is a valid answer.

15. Builds To (Related LeetCode Problems)

LC 231 - Power of Two

↓

LC 326 - Power of Three

↓

LC 342 - Power of Four

↓

LC 263 - Ugly Number

↓

LC 204 - Count Primes

These develop number theory skills.

16. Alternate Approaches + Comparison



Repeated Division	$O(\log n)$	$O(1)$	Best
Recursion	$O(\log n)$	$O(\log n)$	Good
Logarithms	$O(1)$	$O(1)$	Risky
Largest Power Trick	$O(1)$	$O(1)$	Advanced

Approach 1: Repeated Division (Chosen)

Most intuitive.

Most interview-friendly.

Approach 2: Recursion

def isPowerOfThree(n):

if n == 1:

return True

if n <= 0 or n % 3:

return False

return isPowerOfThree(n//3)

Approach 3: Largest Power Trick

Largest power of 3 within 32-bit integer:

$1162261467 = 3^{19}$

Then:

return n > 0 and $1162261467 \% n == 0$

Advanced mathematical solution.

17. Why This Solution Works (Short Intuition)

A power of 3 contains only factors of 3.

Repeatedly removing factors of 3 must eventually reduce the number to 1.

If any other factor exists, the process stops before reaching 1.

18. Variations / Follow-Up Questions

Follow-Up 1

Check Power of Four.



Related:

LC 342

Follow-Up 2

Check Power of Two.

Related:

LC 231

Follow-Up 3

Find the smallest power of 3 greater than N.

Follow-Up 4

Return the exponent.

Example:

$81 \rightarrow 4$

because:

$81 = 3^4$

Interview Takeaway

The key pattern is:

Repeated Division

Whenever you hear:

Power of X

Factorization

Divisibility

think:

Keep dividing by the base

If you eventually reach:

1

the number is a valid power of that base.